POSTMAN CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. OVERVIEW

Installation

Postman API style/protocol for service communication
Key principles and design philosophy
Compare to REST, SOAP, GraphQL alternatives

```
# Typical request
curl -X POST https://api.example.com/graphql \
-H 'Content-Type: application/json' \
-d '{"query": "{ users { id name } }"}'
```

02. CORE CONCEPTS

Architecture

Schema: defines types, queries, mutations
Operations: Query (read), Mutation (write), Subscription (real-time)
Resolver: function that returns data for a field

```
# Schema example:
type User { id: ID!, name: String!, email: St...
type Query { user(id: ID!): User, users: [Use...
```

03. SETUP & SERVER

YAML / Config

```
npm install apollo-server graphql
const { ApolloServer } = require('@apollo/ser...
const server = new ApolloServer({ typeDefs, r...
await server.listen({ port: 4000 })
# GraphQL Playground at http://localhost:4000
```

04. QUERIES & MUTATIONS

Workflow

```
# Query
query { user(id: "1") { name email } }
# Mutation
mutation { createUser(name: "Alice", email: "...
# Variables
query GetUser($id: ID!) { user(id: $id) { nam...
{ "id": "123" }
```

05. RESOLVERS

CRUD API

```
const resolvers = {
Query: {
user: (_, { id }, ctx) => ctx.db.users.findBy...
users: (_, __, ctx) => ctx.db.users.findAll(),
},
Mutation: {
createUser: (_, args, ctx) => ctx.db.users.cr...
},
}
```

06. AUTH & SECURITY

Integration

Add auth in context or middleware

```
const server = new ApolloServer({
context: async ({ req }) => ({
user: await authenticate(req.headers.authoriz...
db,
}),
})
```

Validate and sanitize all inputs
Rate limit and depth-limit queries

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. ERROR HANDLING

Hardening

```
throw new UserInputError('Invalid input', { f...
throw new AuthenticationError('Not authentica...
throw new ForbiddenError('Not authorized')
```

Errors returned in errors[] array alongside data
Use formatError to sanitize error messages in production

08. SUBSCRIPTIONS

Observability

```
type Subscription { messageAdded: Message! }
Subscription: {
messageAdded: {
subscribe: () => pubsub.asyncIterator(['MESSA...
}
}
pubsub.publish('MESSAGE_ADDED', { messageAdde...
```

Requires WebSocket transport (subscriptions-transport-ws or graphql-ws)",

09. PERFORMANCE

Optimization

N+1 problem: use DataLoader for batching

```
const userLoader = new DataLoader(ids => batc...
Query.user = (_, { id }) => userLoader.load(i...
```

Persisted queries: cache query document hash
Response caching: @cacheControl directive
Field-level tracing for performance analysis

10. TOOLING

Unit Tests

```
GraphQL Playground / Apollo Studio  interacti...
graphql-codegen    generate types from schema
graphql-inspector  schema diff/validation
Apollo Client      frontend integration
urql               lightweight GraphQL client
graphql-shield     permission layer
```

11. TESTING

Commands

```
# Apollo Server integration test
const { query } = createTestClient(server)
const { data } = await query({ query: GET_USE...
expect(data.user.name).toBe('Alice')
```

Test resolvers in isolation with mock context
Use Jest or Vitest as test runner

12. COMMON GOTCHAS

Warning

N+1 queries: always use DataLoader for relations
Query depth limiting: prevent DoS attacks
Don't expose internal error details in production
Schema-first vs code-first: choose one approach
Subscriptions need separate WebSocket transport setup